Making LLMs work on analog optical chips:

ADC-Aware Quantization of Large Language Models for In-Memory Computing

Alina Burykina

Constructor University | May 2026

LLM inference is energetically expensive

LLM inference is energetically expensive

Analog optical hardware is $\sim 10^3 \times$ more energy-efficient for linear operations

LLM inference is energetically expensive

Analog optical hardware is $\sim 10^3\times$ more energy-efficient for linear operations

Collaboration with an optical hardware group at Oxford

How analog optical inference looks

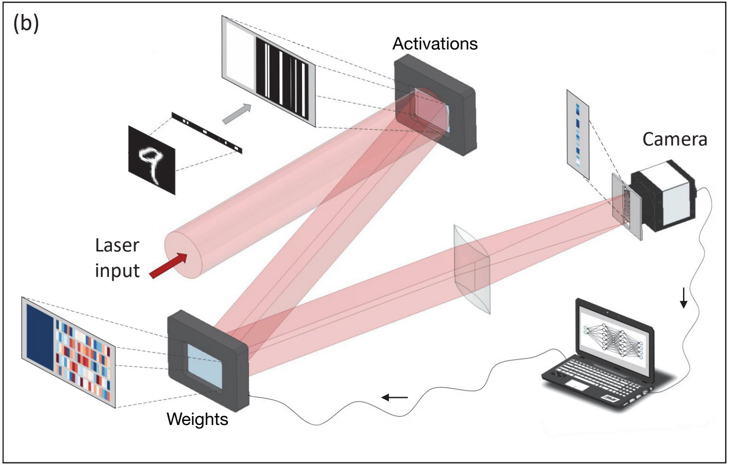
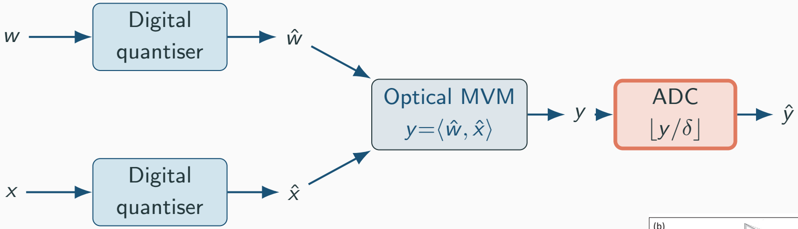
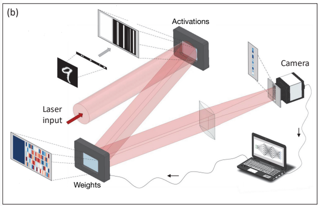


Figure adapted from Spall et al., *Hybrid training of optical neural networks*, arXiv:2203.11207

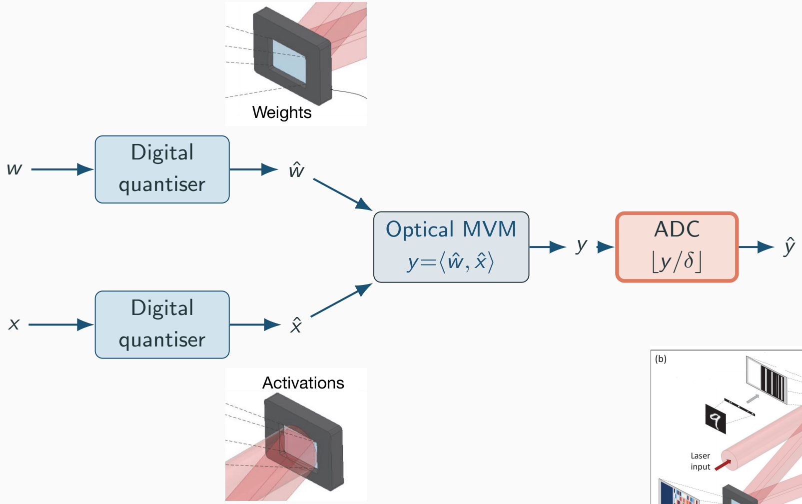
Math model of an optical linear layer

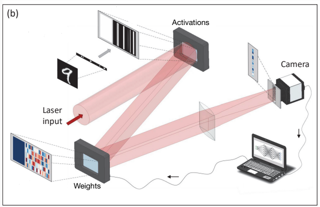




* *MVM — matrix-vector multiplication.*
* *ADC — analog-to-digital converter.*

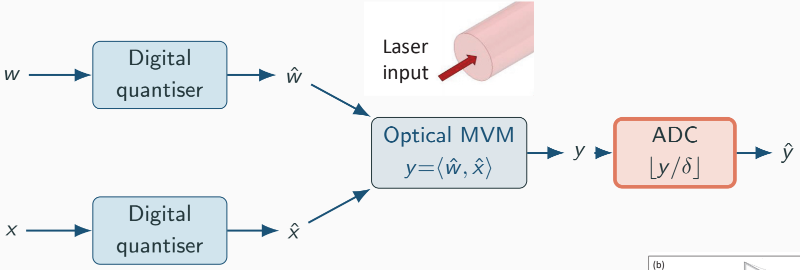
Math model of an optical linear layer

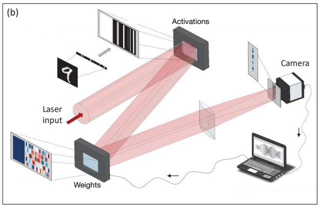




* *MVM — matrix-vector multiplication.*
* *ADC — analog-to-digital converter.*

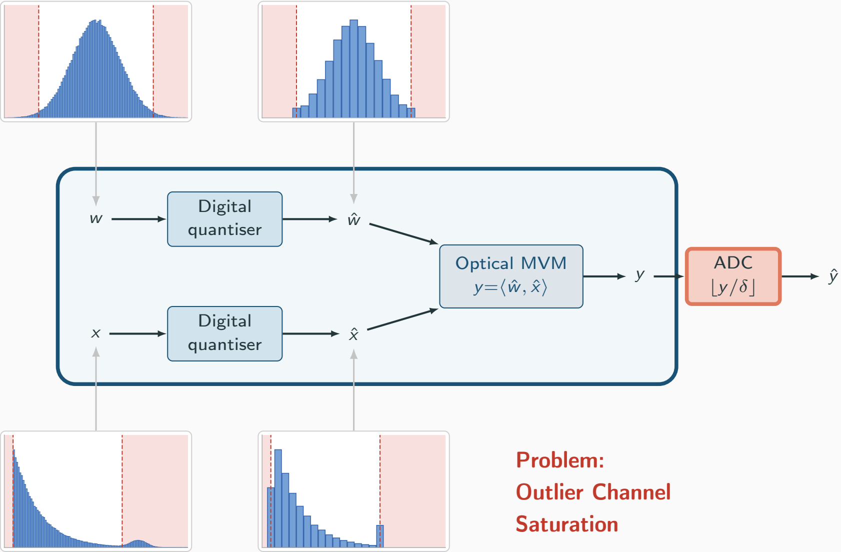
Math model of an optical linear layer





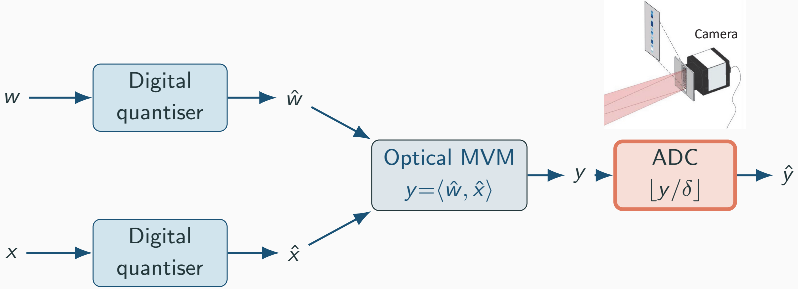
* *MVM — matrix-vector multiplication.*
* *ADC — analog-to-digital converter.*

Digital Quantisation



Problem:
Outlier Channel Saturation

Math model of an optical linear layer

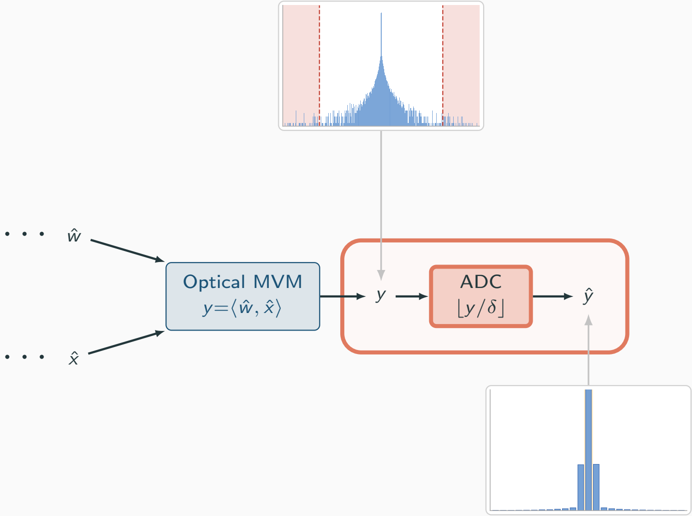


* *MVM — matrix-vector multiplication.*
* *ADC — analog-to-digital converter.*

ADC: Reading the Analog Accumulation

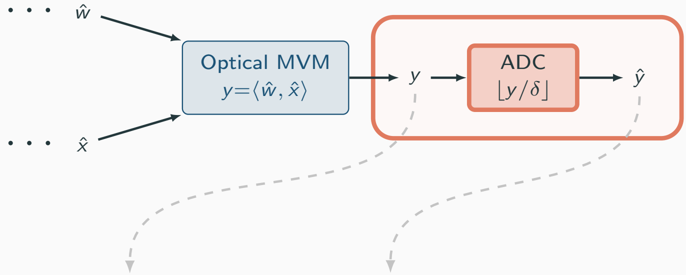
Problems:

1. δ is fixed by the hardware
2. Quantization Aware Training (QAT) is not an option



Thesis goal

Evaluating model perplexity (PPL) for FlatQuant with 4-bit weights and activations.



PPL no ADC	PPL with ADC
15.41	**2354**

THESIS GOAL

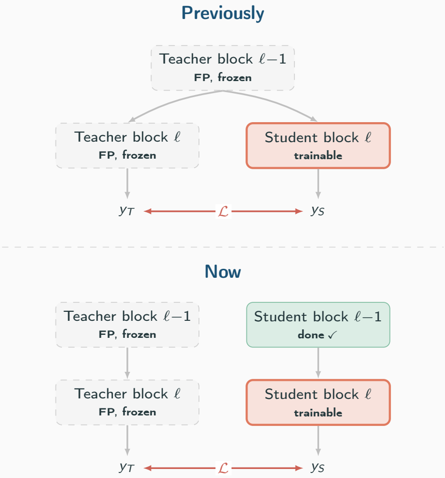
Make Post Train Quantization (PTQ) work for optical chips

Our Extension I: Cross-block Propagation

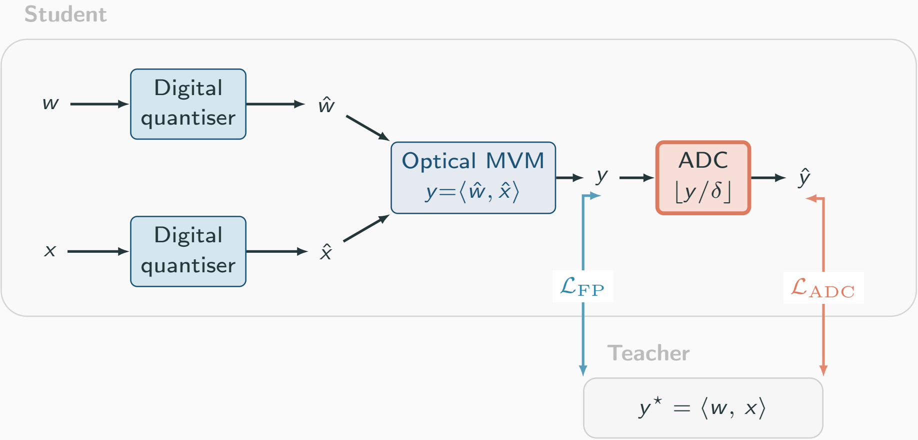
$$y = xW^\top = \underbrace{(xT^{-1})}_{\tilde{x}}\underbrace{(TW^\top)}_{\tilde{W}^\top}.$$

Previously:
Each transformer block is calibrated independently $\Rightarrow$ mismatch.

Now: error accumulates the same way during train and inference.



Our Extension II: α-Mixed Objective

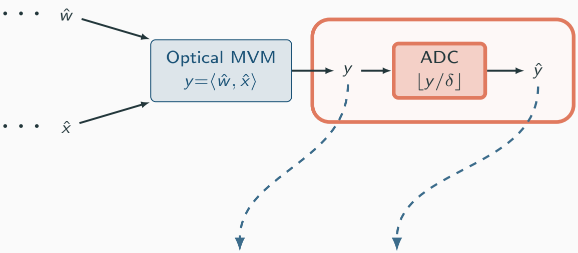


$$\mathcal{L}(\alpha) = (1-\alpha)\,\mathcal{L}_{\mathrm{FP}} + \alpha\,\mathcal{L}_{\mathrm{ADC}}$$

Our Extension III: Selective Diagonal Placement

$$D = \mathrm{diag}(d_1, \ldots, d_c), \qquad \tilde{T} = T \cdot D$$

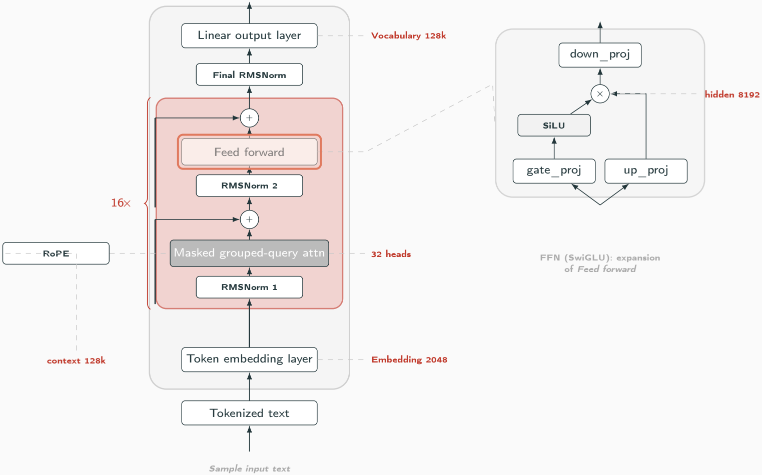
- `diag_attn`: before q/k/v/o projections
  attention-head variance
- `diag_mlp`: before gate/up/down
  SwiGLU outliers



α-mix base +	no-ADC PPL	ADC PPL
— (no diag)	24.24±0.6	31.22±0.9
`diag_mlp` only	19.37±0.4	31.65±1.0
`diag_attn` only	24.98±0.6	29.41±0.8
both diagonals	20.23±0.4	27.56±0.7

Ablation (Llama-3.2-1B, w4a4, α=0.5, ADC PPL↓).

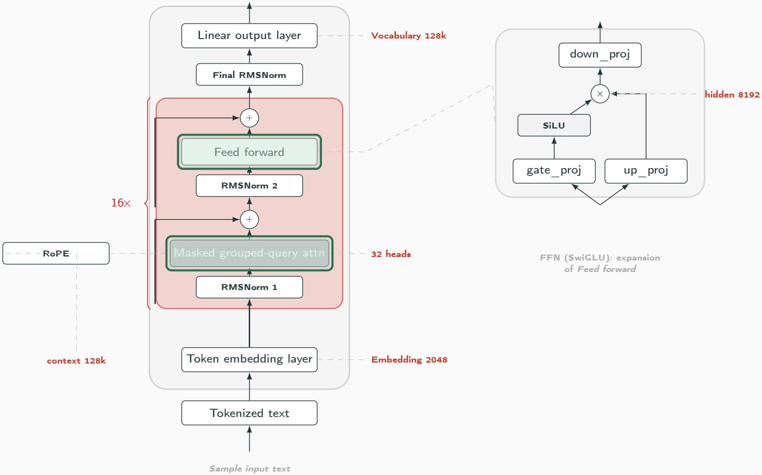
Staged Training: What Is Learnt, When



Stage A | MLP-first

Freeze attention. Train $T_{g,u,d}$ and $D^{\mathrm{m}}_{g,u}$ with $\alpha{=}0$ — clean warmup.

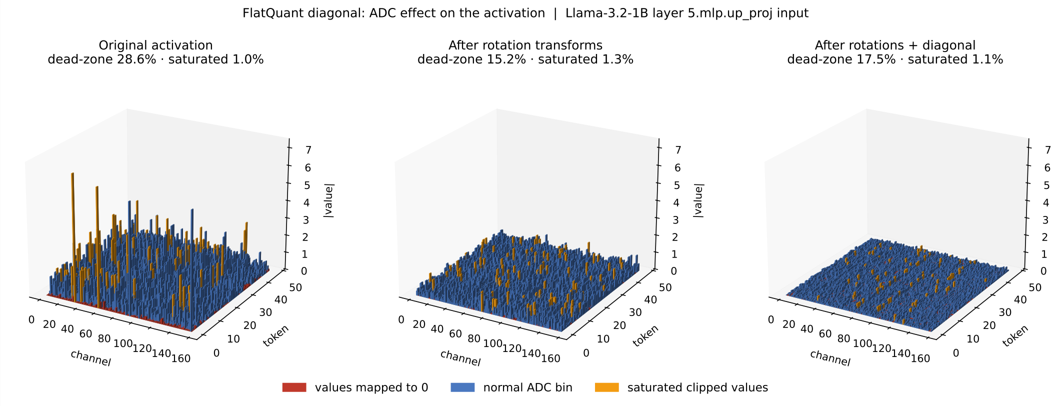
Staged Training: What Is Learnt, When



Stage B | full block

Unfreeze attention. Add $T_{q,k,v,o}$ and $D^{a}_{q,k,v}$. Turn on α=0.5.

Activation Shape vs ADC Effect



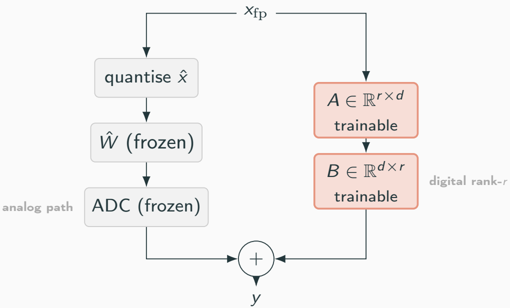
PTQ Results on WikiText-2

Method	**bits (w / a)**	$\mathrm{PPL}_{\text{no ADC}}$	$\mathrm{PPL}_{\text{ADC}}$
FP baseline	16 / 16	8.68	—
FlatQuant	4 / 4	15.41	2354
our setup	4 / 4	**17.83**	**26.66**

PTQ reduces the collapse, but plateaus far from full precision → need a small trainable correction.

Method II: Post-ADC Residual LoRA

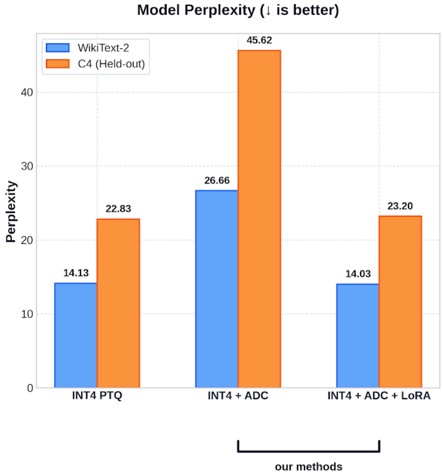
$$y \;=\; \underbrace{\mathrm{ADC}_{\mathrm{frozen}}(\hat{x})}_{\text{quantised path}} \;+\; \underbrace{\tfrac{\alpha_{\mathrm{LoRA}}}{r}\, B\big(A(x_{\mathrm{fp}})\big)}_{\text{residual correction, rank } r}$$

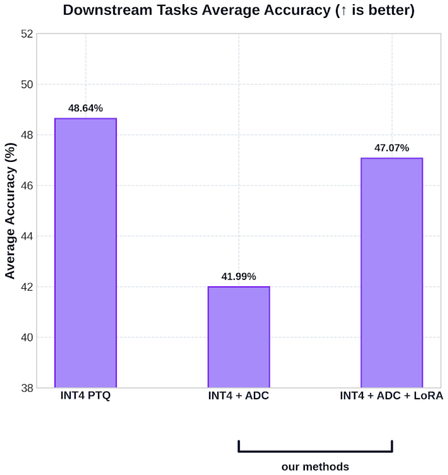


Loss: distill from full-precision teacher

$$\mathcal{L}_{\mathrm{LoRA}} \;=\; \mathrm{CE}(\hat{p}, y) \;+\; \lambda\,\mathrm{KL}(\hat{p} \,\|\, p_{\mathrm{FP}})$$

Final Results on Llama-3.2-1B





Takeaways

1. **First successful PTQ adaptation for optical chips.**
2. **Resolved error accumulation.**
3. **Optimized calibration.**
4. **Broken pure PTQ plateau.**

LoRA Ablations and Overhead

Ablations (Llama-3.2-1B, INT4, $\mathrm{PPL}_{\mathrm{ADC}}\downarrow$)
Base PTQ: staged INT4 ($\mathrm{PPL}_{\mathrm{ADC}} \approx 27.5$).

Axis	Setting	$\mathrm{PPL}_{\mathrm{ADC}}$
Rank r	r=1	15.90
	r=4	15.57
	r=8	**15.24**
Loss	CE only	15.57
	CE + KL	**14.33**
Layers	last 8 only	20.08
	first 8	16.64
	all 16	**15.57**
Placement	Pre-ADC	105.6
	Post-ADC	**15.57**
Coverage	down + o (CE + KL)	14.33
	all 7 (CE + KL)	**14.03**

Overhead on Llama-3.2-1B
(d=2048, 16 blocks, r=4, all 7 proj.)

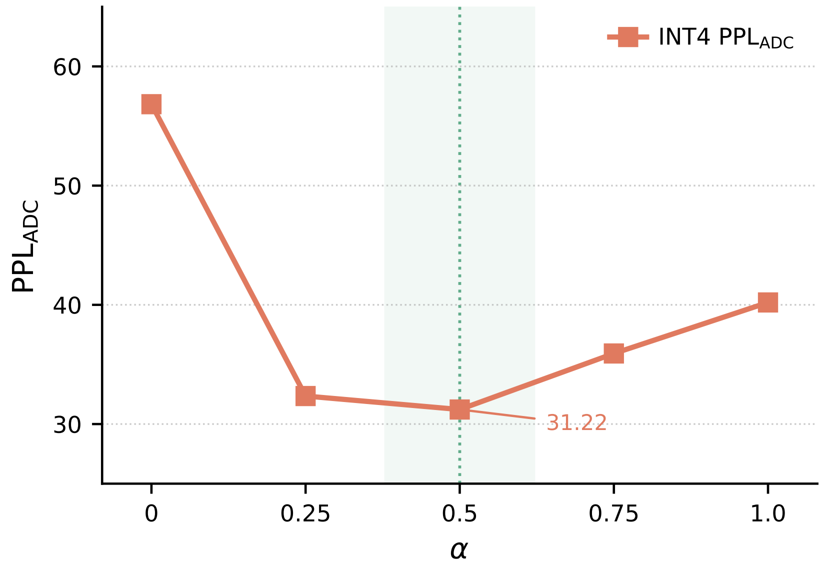
total LoRA params	~2.8 M
as fraction of FP weights	**0.22%**
FP-teacher forward	1× cached
student forward + back	~5× FP MAC
training (1 GPU, 5 ep.)	~45 min
inference MAC overhead	< 0.5%

Backup: Final Results on Llama-3.2-1B

	PPL (↓)		Accuracy % (↑)								
Method	**Wiki**	**C4**	**HSwag**	**MMLU**	**WinGr**	**ARC-E**	**ARC-C**	**PIQA**	**OBQA**	**BoolQ**	**Avg**
bfloat16	**8.68**	**13.13**	**64.19**	**32.15**	63.06	61.83	36.77	74.70	37.40	63.91	**54.25**
INT8 PTQ	8.69	13.15	64.17	31.89	**63.38**	**69.82**	**39.16**	**75.79**	**39.20**	**64.22**	**55.95**
INT8 + ADC PTQ	16.85	29.56	45.04	26.83	56.59	48.44	28.84	64.36	30.00	59.63	44.97
INT4 PTQ	14.13	22.83	51.98	24.86	55.25	60.44	33.02	69.48	32.00	62.11	48.64
INT4 + ADC PTQ	26.66	45.62	40.03	24.41	53.43	42.63	26.02	61.70	27.00	60.67	41.99
INT4 + ADC + LoRA	**14.03**	**23.20**	**49.96**	**25.72**	**53.67**	**54.63**	**29.27**	**68.06**	**34.60**	60.67	**47.07**

PPL lower is better; accuracy higher is better. **Avg** = unweighted mean of the 8 lm-evaluation-harness accuracies. Rows without "+ ADC" are integer-only baselines (no ADC step); rows with "+ ADC" include the finite-resolution ADC (b_a=8, fixed δ).

Backup: α-Sweep — α=0.5 Wins



Backup: Rotation Line of Work

The rotation line of work.

Insert an orthogonal T so that $WT^{-1} \cdot Tx = Wx$ and Tx is easier to quantise:

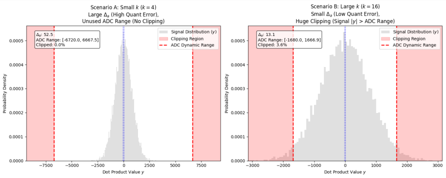
QuaRot	→	SpinQuant	→	**FlatQuant**
Hadamard, fixed		trained rotations		Kron. + per-layer

Why FlatQuant for our work:

- *trainable* — we can add objectives (ADC-aware);
- *Kronecker* ($T = T_1 \otimes T_2$) — memory & compute $\mathcal{O}(\sqrt{d})$;
- per-projection rotations fold naturally into block-wise calibration.

Why ADC is Hard for LLMs: the Outlier Dilemma

LLM activations are heavy-tailed. A tiny fraction of channels carry magnitudes 10–100× larger than the rest *(Dettmers 2022; Xiao 2023)*.



Llama-3.2-1B `q_proj` dot-product distribution, two ADC step sizes

Why we can't just pick a good k:

Large k (fine δ)	**Small k (coarse δ)**
outliers saturate the ADC range $\Rightarrow$ heavy *clipping*	outliers stay in range
model loses critical channels	dynamic range wasted on outliers
good ADC SNR on bulk	bulk collapses into *dead zone*
bad downstream accuracy	bad ADC SNR everywhere

Takeaway

Hardware-fixed δ + heavy-tailed activations $\Rightarrow$ **no single k works**. The algorithm must reshape x and w *before* they hit the array so that y lands in the ADC's usable range.

Backup: Complete INT8 Results (v2)

Setup: Llama-3.2-1B, w8a8, 128 calibration samples. Testing two ideas on top of baseline FlatQuant: (1) *bin-centre loss* — pushes y_{int} toward ADC bin centres during PTQ; (2) *propagated calibration* — each block sees ADC-quantised inputs from the previous block, matching inference conditions. Key finding: propagation gives a 2× ADC PPL drop; bin-centre loss alone gives nothing.

Experiment	Description	$\mathrm{PPL}_{\mathrm{no\ ADC}}$	$\mathrm{PPL}_{\mathrm{ADC}}$	dead_rate
baseline	FlatQuant w8a8	10.01	28.86	10.3%
center mid	bin-centre $\lambda = 0.1$	10.01	28.86	10.3%
prop	propagated calibration	11.01	**14.55**	10.3%
prop+center	propagated + bin-centre	11.00	**14.41**	10.3%

Backup: Complete INT4 v2 Alpha-Sweep

Setup: Llama-3.2-1B, w4a4, 1024 samples. Sweeping α in the mixed loss $\mathcal{L}(\alpha) = (1-\alpha)\mathcal{L}_{\mathrm{FP}} + \alpha\mathcal{L}_{\mathrm{ADC}}$. $\alpha=0$ = clean FP inputs only; $\alpha=1$ = ADC-noisy inputs only. Gap = ADC PPL / no-ADC PPL. Last two rows add diagonals and staged training on top of $\alpha=0.5$.

Experiment	α	$\mathrm{PPL}_{\mathrm{no\ ADC}}$	$\mathrm{PPL}_{\mathrm{ADC}}$	gap
baseline_1024	0.0	19.80	56.83	$\times$2.87
prop_full_1024	1.0	30.32	40.20	$\times$1.32
propalpha_075_1024	0.75	27.46	35.92	$\times$1.31
propalpha_05_1024	0.5	24.24	**31.22**	$\times$**1.29**
propalpha_025_1024	0.25	21.65	32.35	$\times$1.49
add_diag + α=0.5	0.5	20.23	**27.56**	$\times$1.36
staged_mlp_attn	0.5	**18.50**	**27.60**	$\times$1.49

Backup: Complete v7 LoRA Ablation

Setup: Post-ADC LoRA correction on frozen PTQ model (Llama-3.2-1B, w4a4, staged PTQ base: no-ADC=17.89, ADC=27.55). LoRA adapters trained with CE loss on wikitext2, optionally + KL divergence term. Ablating: target projections (`down+o` vs all 7), rank (1–8), loss (CE vs CE+KL), layer selection (first/last 8), and pre- vs post-ADC placement (pre-ADC fails catastrophically).

Experiment	Targets	Loss	Rank	$\mathrm{PPL}_{\text{no ADC}}$	$\mathrm{PPL}_{\text{ADC}}$
base (no LoRA)	—	—	—	17.89	27.55
r1_down_o	down+o	CE	1	—	15.90
r2_down_o	down+o	CE	2	—	15.60
r4_down_o	down+o	CE	4	18.68	15.57
r8_down_o	down+o	CE	8	—	15.24
r4_down_o_ce_kl	down+o	CE+KL	4	19.54	**14.33**
r4_down_o_first8	down+o	CE	4	18.42	16.64
r4_down_o_last8	down+o	CE	4	18.62	20.08
pre_adc_r4_down_o	down+o	CE	4	123.34	**105.63**
r4_all_ce	all 7	CE	4	17.62	16.68
r4_all_ce_kl	all 7	CE+KL	4	19.13	**14.03**

Backup: v8 Generalisation to C4

Setup: Same PTQ base as v7, LoRA trained on wikitext2 only, evaluated on both wikitext2 and C4. Checks whether the best v7 config (`r4_all_ce_kl`) generalises out-of-domain. Without LoRA, the base model has a large C4 ADC gap (46.40 vs no-ADC 29.41); LoRA closes it to 23.30 on C4 — confirming the correction is not overfitting to the training distribution.

Experiment	Targets	Wiki no-ADC	Wiki ADC	C4 no-ADC	C4 ADC
base (no LoRA)	—	18.49	27.26	29.41	46.40
rank8_down_o (CE)	down+o	18.51	15.47	30.48	29.39
r4_down_o_ce_kl	down+o	19.05	**14.27**	29.37	**23.88**
r4_all_ce_kl	all 7	18.70	**14.03**	29.10	**23.30**

Backup: v9 Data Efficiency (lora_nsamples sweep)

Setup: Best config (`r4_all_ce_kl`, `mvm_limit=256`), PTQ fixed at 1024 samples. Varying how many calibration sequences LoRA sees during fine-tuning (64 $\rightarrow$ 1024). Shows LoRA improves even with 64 samples, but ADC PPL keeps improving up to 1024 — more data helps consistently.

Experiment	`nsamples`	Wiki $\mathrm{PPL}_{\mathrm{no\ ADC}}$	Wiki $\mathrm{PPL}_{\mathrm{ADC}}$	C4 $\mathrm{PPL}_{\mathrm{no\ ADC}}$	C4 $\mathrm{PPL}_{\mathrm{ADC}}$
r4_all_ce_kl_n64	64	18.42	21.07	29.50	35.62
r4_all_ce_kl_n128	128	18.21	18.09	28.97	30.19
r4_all_ce_kl_n256	256	18.66	15.88	28.84	26.13
r4_all_ce_kl_n512	512	17.99	**14.82**	28.20	**24.75**
r4_all_ce_kl_n1024	1024	18.92	**14.06**	28.79	**23.56**

Config: `r4_all_ce_kl`, `mvm_limit=256`. PTQ fixed at 1024 samples.

Backup: v9 KL Weight $\times$ Temperature Sweep

Setup: Base `r4_all_ce_kl` (1024 samples). KL term $\lambda_{\mathrm{KL}} \cdot D_{\mathrm{KL}}(\hat{p}_{\mathrm{ADC}} \,\|\, \hat{p}_{\mathrm{FP}})$ with softmax temperature T. Higher T softens logits before KL, making the teacher signal smoother. λ_{KL}=0.5, T=1 gives the best C4 ADC PPL (23.10); very high λ (2.0/T=4) destabilises training.

Experiment	λ_{KL}	T	Wiki $\mathrm{PPL}_{\mathrm{no\ ADC}}$	Wiki $\mathrm{PPL}_{\mathrm{ADC}}$	C4 $\mathrm{PPL}_{\mathrm{no\ ADC}}$	C4 $\mathrm{PPL}_{\mathrm{ADC}}$
r4_kl025_t1	0.25	1.0	18.77	13.84	29.16	23.23
r4_kl025_t2	0.25	2.0	19.15	14.07	29.03	23.33
r4_kl05_t1	0.50	1.0	18.92	**13.79**	29.69	**23.10**
r4_kl05_t2	0.50	2.0	18.43	14.05	28.54	23.37
r4_kl10_t2	1.00	2.0	18.90	13.94	29.47	23.17
r4_kl20_t4	2.00	4.0	*failed*			

Base: `r4_all_ce_kl` (1024 samples). Best: $\lambda = 0.5$, $T = 1$ — C4 ADC 23.10.

Backup: Stochastic Propagation (v5)

Setup: Instead of a fixed α, sample α_t each batch — either Bernoulli($\{0,1\}$) or Beta$(2,2)$ (concentrated around 0.5). Bernoulli with staged training gives the best no-ADC PPL ever seen (16.94) but a bad ADC gap, because FP-only batches teach nothing about ADC noise. Beta$(2,2)$ $\approx$ deterministic α=0.5 in expectation and achieves the best ADC PPL (27.82).

Experiment	Mode	$\mathrm{PPL}_{\text{no ADC}}$	$\mathrm{PPL}_{\text{ADC}}$	gap
stoch_bern (flat)	Bernoulli	19.25	35.38	×1.84
stoch_beta2 (flat)	Beta(2,2)	22.90	31.40	×1.37
staged + Bernoulli	Bernoulli	**16.94**	32.44	×1.92
staged + Beta(2,2)	Beta(2,2)	17.16	**27.82**	×1.62
deterministic α=0.5	—	24.24	31.22	×1.29

Bernoulli gives best no-ADC but worst ADC gap (hard 0/1 switching provides no ADC signal on FP batches). Beta(2,2) $\approx$ deterministic α=0.5 in expectation.

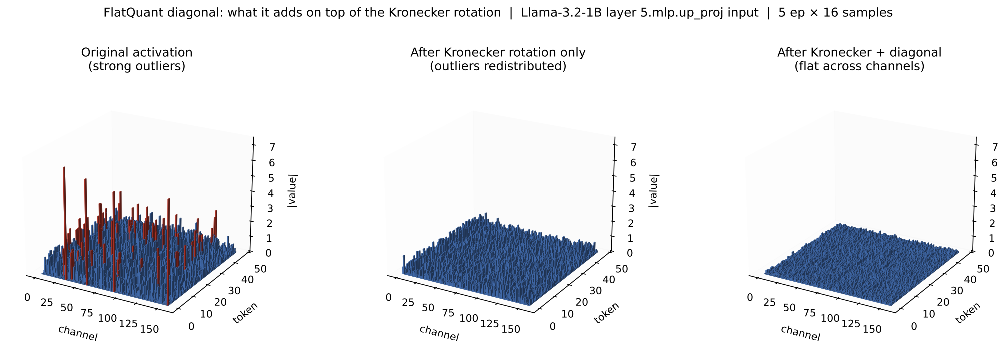
Backup: Trainable Diagonals (FlatQuant Component)

Absorb a trainable diagonal $D = \mathrm{diag}(d_1, \ldots, d_c)$ into the rotation:

$$\tilde{T} = T \cdot D, \qquad d_i > 0$$

Invariance preserved — opposite scaling on the next layer:

$$y = (xD^{-1}T^{-1})\,(TDW^\top).$$



Backup: Staged Training — Why Two Stages?

MLP wants it clean

best $\mathrm{PPL}_{\mathrm{no\ ADC}}$ with `diag_mlp`, α=0 — ADC suffers.

Attention wants ADC noise

best $\mathrm{PPL}_{\mathrm{ADC}}$ with `diag_attn`, $\alpha \approx 0.5$ — no-ADC PPL suffers.

Train sequentially — MLP first, then add attention.

Backup: Hardware Config & Delta Formula

Parameter	Value
b_w (weight bits)	4 / 8
b_x (act. bits)	4 / 8
b_a (ADC bits)	8
`mvm_limit`	256 / 1024
k (ADC parallelism)	16
δ (mvm=256, INT8)	$\approx$ 2016
δ (mvm=256, INT4)	$\approx$ 6.12
δ (mvm=1024, INT8)	$\approx$ 8064
FlatQuant epochs	30
FlatQuant LR	0.005
Cal. samples (INT4)	1024
Cal. samples (INT8)	128

Delta formula ($q = 2^{b-1}-1$):

$$\delta = \frac{2 \cdot \texttt{tile\_in} \cdot q_x \cdot q_w}{2^{b_a} \cdot k}$$

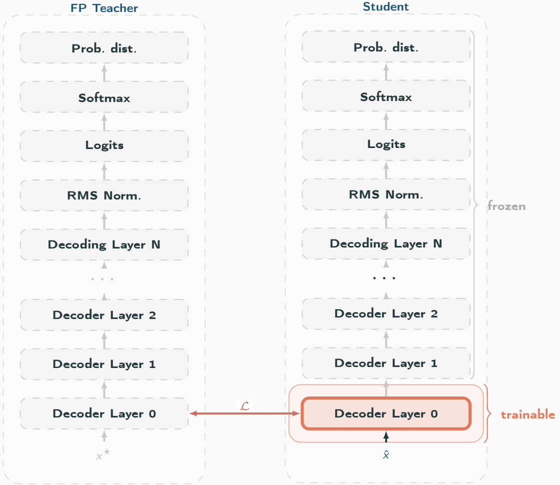
- INT8 (q=127, mvm=256): $\frac{2 \cdot 256 \cdot 127^2}{256 \cdot 16} \approx 2016$
- INT4 (q=7, mvm=256): $\frac{2 \cdot 256 \cdot 7^2}{256 \cdot 16} \approx 6.12$
- INT8, mvm=1024: $\frac{2 \cdot 1024 \cdot 127^2}{256 \cdot 16} \approx 8064$

INT8: useful ADC bins in $\pm\sigma_z \approx$ 15–30 (empirical) — explains why INT8 PTQ alone plateaus.

Backup: Block-wise Calibration (FlatQuant Baseline)

FlatQuant standard. Train *one block at a time*, both teacher and student see the same FP input $x^\star$:

$$\mathcal{L} = \left\| \hat{B}(x^\star) - B_{\mathrm{FP}}(x^\star) \right\|_2^2$$



Backup: FlatQuant — Rotate, Flatten, Calibrate per Block

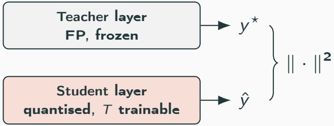
1. **Linear invariance.**

$$y = xW^{\top} = \underbrace{(xT^{-1})}_{\tilde{x}}\underbrace{(TW^{\top})}_{\tilde{W}^{\top}}.$$

2. **Kronecker factorisation.**

$$T = T_1 \otimes T_2, \quad T_1, T_2 \in \mathbb{R}^{\sqrt{d}\times\sqrt{d}}$$

3. **Teacher-student block-wise calibration.**



This Work in Context

Method	Key idea	Gap / limit
FlatQuant	Learned Kronecker-decomposed orthogonal transforms per layer	FP loss only; no ADC signal
OmniQuant / LET	Learnable per-channel equivalent scaling (diagonal)	No ADC-aware objective
QEP / prop. calib.	Feed quantised activations into the next layer's calibration	INT8 only; no mixed-loss
PACT / QDrop	Stochastic / clamped quant-aware training	Full re-training required
This work	ADC-aware PTQ (α-mixed, staged diag) + post-ADC LoRA	—

Key novelty

No prior method directly targets the **post-ADC residual** — our LoRA correction is the first to do so.

Post-ADC LoRA: Parameter and Compute Overhead

Trainable parameter count (r=4, all 7 projections):

Projection	A	B
q_proj	4×2048	2048×4
k_proj	4×2048	512×4
v_proj	4×2048	512×4
o_proj	4×2048	2048×4
gate_proj	4×2048	8192×4
up_proj	4×2048	8192×4
down_proj	4×8192	2048×4

Inference overhead (per forward pass):

- Extra FLOPs per projection: $d_{\mathrm{in}} \cdot r + r \cdot d_{\mathrm{out}} \ll d_{\mathrm{in}} \cdot d_{\mathrm{out}}$
- Example: `q_proj`: 8192+8192 vs. $4M$ **(<0.5% overhead)**
- Memory: 2.82M fp32 params ≈ **11 MB** vs. ∼2 GB base model
- LoRA correction runs on *digital* hardware — does not increase analog area

Bottom line

0.228% extra params, < 0.5% extra FLOPs, correction entirely in digital post-processing. The trade-off is **strongly favourable**.

Thank you.

Questions?

Alina Burykina | Constructor University | April 2026